

10.3 Sorted Maps

The traditional map ADT allows a user to look up the value associated with a given key, but the search for that key is a form known as an *exact search*. In this section, we will introduce an extension known as the *sorted map* ADT that includes all behaviors of the standard map, plus the following:

- `firstEntry()`: Returns the entry with smallest key value (or null, if the map is empty).
- `lastEntry()`: Returns the entry with largest key value (or null, if the map is empty).
- `ceilingEntry(k)`: Returns the entry with the least key value greater than or equal to k (or null, if no such entry exists).
- `floorEntry(k)`: Returns the entry with the greatest key value less than or equal to k (or null, if no such entry exists).
- `lowerEntry(k)`: Returns the entry with the greatest key value strictly less than k (or null, if no such entry exists).
- `higherEntry(k)`: Returns the entry with the least key value strictly greater than k (or null if no such entry exists).
- `subMap( $k_1$ ,  $k_2$ )`: Returns an iteration of all entries with key greater than or equal to k_1 , but strictly less than k_2 .

We note that the above methods are included within the `java.util.NavigableMap` interface (which extends the simpler `java.util.SortedMap` interface).

To motivate the use of a sorted map, consider a computer system that maintains information about events that have occurred (such as financial transactions), with a *time stamp* marking the occurrence of each event. If the time stamps were unique for a particular system, we could organize a map with a time stamp serving as a key, and a record about the event that occurred at that time as the value. A particular time stamp could serve as a reference ID for an event, in which case we can quickly retrieve information about that event from the map. However, the (unsorted) map ADT does not provide any way to get a list of all events ordered by the time at which they occur, or to search for which event occurred closest to a particular time. In fact, hash-based implementations of the map ADT intentionally scatter keys that may seem very “near” to each other in the original domain, so that they are more uniformly distributed in a hash table.

10.3.1 Sorted Search Tables

Several data structures can efficiently support the sorted map ADT, and we will examine some advanced techniques in Section 10.4 and Chapter 11. In this section, we will begin by exploring a simple implementation of a sorted map. We store the map's entries in an array list A so that they are in increasing order of their keys. (See Figure 10.8.) We refer to this implementation as a *sorted search table*.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
2	4	5	7	8	9	12	14	17	19	22	25	27	28	33	37

Figure 10.8: Realization of a map by means of a sorted search table. We show only the keys for this map, so as to highlight their ordering.

As was the case with the unsorted table map of Section 10.1.4, the sorted search table has a space requirement that is $O(n)$. The primary advantage of this representation, and our reason for insisting that A be array-based, is that it allows us to use the *binary search* algorithm for a variety of efficient operations.

Binary Search and Inexact Searches

We originally presented the binary search algorithm in Section 5.1.3, as a means for detecting whether a given target is stored within a sorted sequence. In our original presentation (Code Fragment 5.3 on page 197), a `binarySearch` method returned `true` or `false` to designate whether the desired target was found.

The important realization is that, while performing a binary search, we can instead return the index at or near where a target might be found. During a successful search, the standard implementation determines the precise index at which the target is found. During an unsuccessful search, although the target is not found, the algorithm will effectively determine a pair of indices designating elements of the collection that are just less than or just greater than the missing target.

In Code Fragments 10.11 and 10.12, we present a complete implementation of a class, `SortedTableMap`, that supports the sorted map ADT. The most notable feature of our design is the inclusion of a `findIndex` utility method. This method uses the recursive binary search algorithm, but returns the *index* of the leftmost entry in the search range having key greater than or equal to k ; if no entry in the search range has such a key, we return the index just beyond the end of the search range. By this convention, if an entry has the target key, the search returns the index of that entry. (Recall that keys are unique in a map.) If the key is absent, the method returns the index at which a new entry with that key would be inserted.

```

1  public class SortedTableMap<K,V> extends AbstractSortedMap<K,V> {
2      private ArrayList<MapEntry<K,V>> table = new ArrayList<>();
3      public SortedTableMap() { super(); }
4      public SortedTableMap(Comparator<K> comp) { super(comp); }
5      /** Returns the smallest index for range table[low..high] inclusive storing an entry
6          with a key greater than or equal to k (or else index high+1, by convention). */
7      private int findIndex(K key, int low, int high) {
8          if (high < low) return high + 1;           // no entry qualifies
9          int mid = (low + high) / 2;
10         int comp = compare(key, table.get(mid));
11         if (comp == 0)
12             return mid;                           // found exact match
13         else if (comp < 0)
14             return findIndex(key, low, mid - 1); // answer is left of mid (or possibly mid)
15         else
16             return findIndex(key, mid + 1, high); // answer is right of mid
17     }
18     /** Version of findIndex that searches the entire table */
19     private int findIndex(K key) { return findIndex(key, 0, table.size() - 1); }
20     /** Returns the number of entries in the map. */
21     public int size() { return table.size(); }
22     /** Returns the value associated with the specified key (or else null). */
23     public V get(K key) {
24         int j = findIndex(key);
25         if (j == size() || compare(key, table.get(j)) != 0) return null; // no match
26         return table.get(j).getValue();
27     }
28     /** Associates the given value with the given key, returning any overridden value.*/
29     public V put(K key, V value) {
30         int j = findIndex(key);
31         if (j < size() && compare(key, table.get(j)) == 0)           // match exists
32             return table.get(j).setValue(value);
33         table.add(j, new MapEntry<K,V>(key,value));                 // otherwise new
34         return null;
35     }
36     /** Removes the entry having key k (if any) and returns its associated value. */
37     public V remove(K key) {
38         int j = findIndex(key);
39         if (j == size() || compare(key, table.get(j)) != 0) return null; // no match
40         return table.remove(j).getValue();
41     }

```

Code Fragment 10.11: An implementation of the SortedTableMap class. (Continues in Code Fragment 10.12.) The AbstractSortedMap base class (available online), provides the utility method, compare, based on a given comparator.

```

42  /** Utility returns the entry at index j, or else null if j is out of bounds. */
43  private Entry<K,V> safeEntry(int j) {
44      if (j < 0 || j >= table.size()) return null;
45      return table.get(j);
46  }
47  /** Returns the entry having the least key (or null if map is empty). */
48  public Entry<K,V> firstEntry() { return safeEntry(0); }
49  /** Returns the entry having the greatest key (or null if map is empty). */
50  public Entry<K,V> lastEntry() { return safeEntry(table.size()-1); }
51  /** Returns the entry with least key greater than or equal to given key (if any). */
52  public Entry<K,V> ceilingEntry(K key) {
53      return safeEntry(findIndex(key));
54  }
55  /** Returns the entry with greatest key less than or equal to given key (if any). */
56  public Entry<K,V> floorEntry(K key) {
57      int j = findIndex(key);
58      if (j == size() || ! key.equals(table.get(j).getKey()))
59          j--; // look one earlier (unless we had found a perfect match)
60      return safeEntry(j);
61  }
62  /** Returns the entry with greatest key strictly less than given key (if any). */
63  public Entry<K,V> lowerEntry(K key) {
64      return safeEntry(findIndex(key) - 1); // go strictly before the ceiling entry
65  }
66  public Entry<K,V> higherEntry(K key) {
67      /** Returns the entry with least key strictly greater than given key (if any). */
68      int j = findIndex(key);
69      if (j < size() && key.equals(table.get(j).getKey()))
70          j++; // go past exact match
71      return safeEntry(j);
72  }
73  /** support for snapshot iterators for entrySet() and subMap() follow
74  private Iterable<Entry<K,V>> snapshot(int startIndex, K stop) {
75      ArrayList<Entry<K,V>> buffer = new ArrayList<>();
76      int j = startIndex;
77      while (j < table.size() && (stop == null || compare(stop, table.get(j)) > 0))
78          buffer.add(table.get(j++));
79      return buffer;
80  }
81  public Iterable<Entry<K,V>> entrySet() { return snapshot(0, null); }
82  public Iterable<Entry<K,V>> subMap(K fromKey, K toKey) {
83      return snapshot(findIndex(fromKey), toKey);
84  }
85  }

```

Code Fragment 10.12: An implementation of the SortedTableMap class (continued from Code Fragment 10.11).

Analysis

We conclude by analyzing the performance of our SortedTableMap implementation. A summary of the running times for all methods of the sorted map ADT (including the traditional map operations) is given in Table 10.3. It should be clear that the size, firstEntry, and lastEntry methods run in $O(1)$ time, and that iterating the keys of the table in either direction can be performed in $O(n)$ time.

The analysis for the various forms of search all depend on the fact that a binary search on a table with n entries runs in $O(\log n)$ time. This claim was originally shown as Proposition 5.2 in Section 5.2, and that analysis clearly applies to our findIndex method as well. We therefore claim an $O(\log n)$ worst-case running time for methods get, ceilingEntry, floorEntry, lowerEntry, and higherEntry. Each of these makes a single call to findIndex, followed by a constant number of additional steps to determine the appropriate answer based on the index. The analysis of subMap is a bit more interesting. It begins with a binary search to find the first item within the range (if any). After that, it executes a loop that takes $O(1)$ time per iteration to gather subsequent values until reaching the end of the range. If there are s items reported in the range, the total running time is $O(s + \log n)$.

In contrast to the efficient search operations, update operations for a sorted table may take considerable time. Although binary search can help identify the index at which an update occurs, both insertions and deletions require, in the worst case, that linearly many existing elements be shifted in order to maintain the sorted order of the table. Specifically, the potential call to table.add from within put and table.remove from within remove lead to $O(n)$ worst-case time. (See the discussion of corresponding operations of the ArrayList class in Section 7.2.)

In conclusion, sorted tables are primarily used in situations where we expect many searches but relatively few updates.

Method	Running Time
size	$O(1)$
get	$O(\log n)$
put	$O(n)$; $O(\log n)$ if map has entry with given key
remove	$O(n)$
firstEntry, lastEntry	$O(1)$
ceilingEntry, floorEntry, lowerEntry, higherEntry	$O(\log n)$
subMap	$O(s + \log n)$ where s items are reported
entrySet, keySet, values	$O(n)$

Table 10.3: Performance of a sorted map, as implemented with SortedTableMap. We use n to denote the number of items in the map at the time the operation is performed. The space requirement is $O(n)$.

10.3.2 Two Applications of Sorted Maps

In this section, we explore applications in which there is particular advantage to using a *sorted* map rather than a traditional (unsorted) map. To apply a sorted map, keys must come from a domain that is totally ordered. Furthermore, to take advantage of the inexact or range searches afforded by a sorted map, there should be some reason why nearby keys have relevance to a search.

Flight Databases

There are several websites on the Internet that allow users to perform queries on flight databases to find flights between various cities, typically with the intent to buy a ticket. To make a query, a user specifies origin and destination cities, a departure date, and a departure time. To support such queries, we can model the flight database as a map, where keys are Flight objects that contain fields corresponding to these four parameters. That is, a key is a tuple

$$k = (\text{origin}, \text{destination}, \text{date}, \text{time}).$$

Additional information about a flight, such as the flight number, the number of seats still available in first (F) and coach (Y) class, the flight duration, and the fare, can be stored in the value object.

Finding a requested flight is not simply a matter of finding an exact match for a requested query. Although a user typically wants to exactly match the origin and destination cities, he or she may have flexibility for the departure date, and certainly will have some flexibility for the departure time on a specific day. We can handle such a query by ordering our keys lexicographically. Then, an efficient implementation for a sorted map would be a good way to satisfy users' queries. For instance, given a user query key k , we could call `ceilingEntry( $k$ )` to return the first flight between the desired cities, having a departure date and time matching the desired query or later. Better yet, with well-constructed keys, we could use `subMap( $k_1$ ,  $k_2$ )` to find all flights within a given range of times. For example, if $k_1 = (\text{ORD}, \text{PVD}, \text{05May}, \text{09:30})$, and $k_2 = (\text{ORD}, \text{PVD}, \text{05May}, \text{20:00})$, a respective call to `subMap( $k_1$ ,  $k_2$ )` might result in the following sequence of key-value pairs:

```
(ORD, PVD, 05May, 09:53) : (AA 1840, F5, Y15, 02:05, $251),  
(ORD, PVD, 05May, 13:29) : (AA 600, F2, Y0, 02:16, $713),  
(ORD, PVD, 05May, 17:39) : (AA 416, F3, Y9, 02:09, $365),  
(ORD, PVD, 05May, 19:50) : (AA 1828, F9, Y25, 02:13, $186)
```

Maxima Sets

Life is full of trade-offs. We often have to trade off a desired performance measure against a corresponding cost. Suppose, for the sake of an example, we are interested in maintaining a database rating automobiles by their maximum speeds and their cost. We would like to allow someone with a certain amount of money to query our database to find the fastest car they can possibly afford.

We can model such a trade-off problem as this by using a key-value pair to model the two parameters that we are trading off, which in this case would be the pair (cost, speed) for each car. Notice that some cars are strictly better than other cars using this measure. For example, a car with cost-speed pair (30000, 100) is strictly better than a car with cost-speed pair (40000, 90). At the same time, there are some cars that are not strictly dominated by another car. For example, a car with cost-speed pair (30000, 100) may be better or worse than a car with cost-speed pair (40000, 120), depending on how much money we have to spend. (See Figure 10.9.)

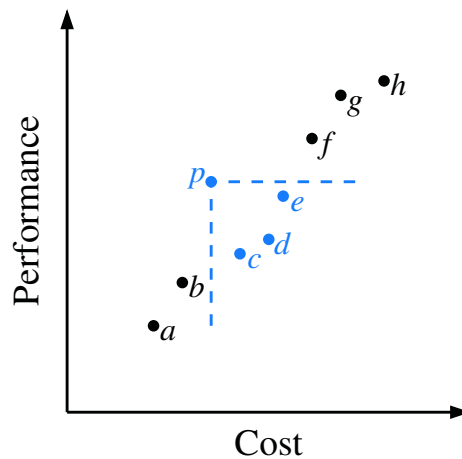


Figure 10.9: Illustrating the cost-performance trade-off with pairs represented by points in the plane. Notice that point p is strictly better than points c , d , and e , but may be better or worse than points a , b , f , g , and h , depending on the price we are willing to pay. Thus, if we were to add p to our set, we could remove the points c , d , and e , but not the others.

Formally, we say a cost-performance pair (a, b) **dominates** pair $(c, d) \neq (a, b)$ if $a \leq c$ and $b \geq d$, that is, if the first pair has no greater cost and at least as good performance. A pair (a, b) is called a **maximum** pair if it is not dominated by any other pair. We are interested in maintaining the set of maxima of a collection of cost-performance pairs. That is, we would like to add new pairs to this collection (for example, when a new car is introduced), and to query this collection for a given dollar amount, d , to find the fastest car that costs no more than d dollars.

Maintaining a Maxima Set with a Sorted Map

We can store the set of maxima pairs in a sorted map so that the cost is the key field and performance (speed) is the value. We can then implement operations $\text{add}(c, p)$, which adds a new cost-performance entry (c, p) , and $\text{best}(c)$, which returns the entry having best performance of those with cost at most c . Code Fragment 10.13 provides an implementation of such a `CostPerformanceDatabase` class.

```

1  /** Maintains a database of maximal (cost,performance) pairs. */
2  public class CostPerformanceDatabase {
3
4      SortedMap<Integer,Integer> map = new SortedTableMap<>();
5
6      /** Constructs an initially empty database. */
7      public CostPerformanceDatabase() { }
8
9      /** Returns the (cost,performance) entry with largest cost not exceeding c.
10       * (or null if no entry exist with cost c or less).
11       */
12     public Entry<Integer,Integer> best(int cost) {
13         return map.floorEntry(cost);
14     }
15
16     /** Add a new entry with given cost c and performance p. */
17     public void add(int c, int p) {
18         Entry<Integer,Integer> other = map.floorEntry(c); // other is at least as cheap
19         if (other != null && other.getValue() >= p) // if its performance is as good,
20             return; // (c,p) is dominated, so ignore
21         map.put(c, p); // else, add (c,p) to database
22         // and now remove any entries that are dominated by the new one
23         other = map.higherEntry(c); // other is more expensive than c
24         while (other != null && other.getValue() <= p) { // if not better performance
25             map.remove(other.getKey()); // remove the other entry
26             other = map.higherEntry(c);
27         }
28     }
29 }

```

Code Fragment 10.13: An implementation of a class maintaining a set of maximal cost-performance entries using a sorted map.

Unfortunately, if we implement the sorted map using the `SortedTableMap` class, the add behavior has $O(n)$ worst-case running time. If, on the other hand, we implement the map using a skip list, which we next describe, we can perform $\text{best}(c)$ queries in $O(\log n)$ expected time and $\text{add}(c, p)$ updates in $O((1 + r) \log n)$ expected time, where r is the number of points removed.